

Let me read the current cookie config and CORS config so the test plan references your actual code, not assumptions.

Now I have the full picture. Here's your test plan, precisely mapped to your actual code.

Auth Refactor — Verification Test Plan

A. Local Dev Verification (localhost:5173 → localhost:3000)

A1. Fresh login sets cookie + memory token

Detail	Value
Action	Submit login form with valid active user
Requests triggered	POST supabase.co/auth/v1/token (by Supabase JS), then POST localhost:3000/api/auth/set-refresh , then GET /api/webapp/users/profiles/me
Inspect: Network tab	/api/auth/set-refresh response has Set-Cookie: sb_refresh=...; HttpOnly; Path=/api/auth; SameSite=Lax (no Secure in dev)
Inspect: Application > Cookies	sb_refresh cookie exists for localhost:3000, marked HttpOnly
Inspect: Application > Local Storage	NO sb-* keys (confirms persistSession: false)
Inspect: Console	No errors; navigate to /dashboard
Backend log to add	In setRefresh : console.log("[auth] set-refresh called, cookie set");

A2. Protected route succeeds

Detail	Value
Request	GET /api/webapp/users/profiles/me

Detail	Value
Header	Authorization: Bearer <token> (attached by axios interceptor)
Expected	200, body has { id, email, status, ... }
Backend log	In authJWT : console.log("[auth] JWKS verified, sub:", payload.sub);

A3. No token in storage

Detail	Value
Check	DevTools → Application → Local Storage + Session Storage for localhost:5173
Expected	Zero Supabase-related keys. Access token only exists in the JS module variable.

A4. Page reload recovers session

Detail	Value
Action	Press F5 on /dashboard
Request triggered	POST /api/auth/refresh (by App.jsx bootstrap)
Expected	200, body { access_token, expires_in } ; app renders dashboard, NOT login
Inspect: Network	Request includes cookie sb_refresh=... ; response has new Set-Cookie (rotated)
Backend log	In refresh : console.log("[auth] refresh OK, rotated cookie");

A5. Access token expiry → silent refresh → retry

Detail	Value
Simulate	In axios.js , temporarily replace setAccessToken(session.access_token) with setAccessToken("expired.fake.token") after login, then click any nav item

Detail	Value
Expected	First API call returns 401 → interceptor calls POST /api/auth/refresh → gets new token → retries original request → 200
Inspect: Network	You'll see the failed request (401), then /auth/refresh (200), then the retried request (200)
Backend log	authJWT logs " Invalid or expired token " once, then " JWKS verified " on retry

A6. Revoked/invalid refresh → forced logout

Detail	Value
Simulate	DevTools → Application → Cookies → edit sb_refresh value to garbage → reload page
Expected	POST /api/auth/refresh returns 401; cookie cleared; redirect to /login
Inspect: Network	/auth/refresh response: 401 { error: "..."} + Set-Cookie: sb_refresh=; Max-Age=0
No infinite loop	Confirm only 1 refresh call (axios skips retry because isRefreshCall check matches)

A7. Logout clears cookie + blocks further calls

Detail	Value
Action	Call api.post("auth/logout") (or add a logout button that does this)
Expected	200 { ok: true } , Set-Cookie: sb_refresh=; Path=/api/auth; Max-Age=0
After logout	Any GET /api/webapp/users/profiles/me → 401 (no Bearer); POST /api/auth/refresh → 401 (no cookie)

Detail	Value
Redirect	window.location.href = "/login" fires

A8. Multi-tab: Tab A logs out, Tab B fails safely

Detail	Value
Setup	Open two tabs, both logged in
Action	Tab A: logout
Tab B action	Click any protected nav item
Expected in Tab B	API call fails 401 → refresh attempt → 401 (cookie gone) → redirect /login
No infinite loop	Single refresh attempt only

A9. Account status: [on_hold](#) user login

Detail	Value
Action	Log in with a user whose profile status = "on_hold"
Expected	Supabase sign-in succeeds (Supabase doesn't know about your status), set-refresh succeeds, GET /profiles/me returns profile with status: "on_hold" → Login.jsx shows: "Your account is currently on hold..." → calls /auth/logout → clears token

A10. Account status: [inactive](#) user login

| Same as A9 but | Message: "Your account has been deactivated..." |

A11. CSRF: Origin check on refresh

Detail	Value
Postman (no Origin)	POST /api/auth/refresh with cookie → 200 (dev allows missing origin)

Detail	Value
Curl with bad Origin	curl -H "Origin: https://evil.com" -X POST localhost:3000/api/auth/refresh --cookie "sb_refresh=..." → 403 { error: "Forbidden origin" }

A12. CORS correctness

Detail	Value
Check	OPTIONS /api/auth/refresh from browser
Expected headers	Access-Control-Allow-Origin: http://localhost:5173, Access-Control-Allow-Credentials: true, Access-Control-Allow-Headers: Content-Type, Authorization
Verify	No * in Allow-Origin (would break credentials)

B. Local "Prod-like" Verification

This phase catches cookie/HTTPS issues *before* deploying.

B1. Verify [NODE_ENV=production](#) cookie behavior

-
-
-
-

Or add a temp route that returns [refreshCookieOpts\(\)](#). Confirm: [secure: true](#), [sameSite: "lax"](#), [httpOnly: true](#), [path: "/api/auth"](#).

B2. Verify [assertOrigin](#) rejects missing Origin in prod

-
-
-
-

B3. Cross-origin cookie decision

Question	Answer determines your config
Are Vercel frontend and Railway backend on the same domain ?	If NO (different domains), you MUST change sameSite to "none" and secure to true. If YES (reverse proxy), keep "lax".

Your current config has [sameSite: "lax"](#). This works only if frontend and backend share the same registrable domain. If Vercel = [app.example.com](#) and Railway = [api.example.com](#), Lax works. If they're on completely different domains (e.g., [app.vercel.app](#) vs [backend.railway.app](#)), you need SameSite=None; Secure.

C. Staging Deployment Verification (Vercel + Railway)

Pre-flight: Environment variables

Service	Required Vars
Railway	SUPABASE_URL , SUPABASE_SERVICE_ROLE_KEY , SUPABASE_ANON_KEY , SUPABASE_API_KEY , NODE_ENV=production
Vercel	VITE_SUPABASE_URL , VITE_SUPABASE_ANON_KEY
Frontend	axios.baseURL must point to Railway URL (not localhost:3000)
Backend	allowedOrigins in both server.js CORS and authController.js must include Vercel domain

C1–C12: Repeat A1–A12 against deployed URLs

Key differences to watch:

Step	Localhost	Deployed
Cookie Secure	false	must be true

Step	Localhost	Deployed
Cookie SameSite	lax works	lax only if same domain; none if cross-domain
CORS origin	http://localhost:5173	https://<vercel-domain>
assertOrigin allowedOrigins	http://localhost:5173	must include https://<vercel-domain>
JWKS fetch	Works from localhost	Ensure Railway can reach supabase.co (no firewall/egress block)

Failure Mode Matrix

#	Symptom	Likely Cause	Confirm	Fix
1	Cookie never set (Application tab empty)	credentials: "include" missing on frontend, or CORS missing credentials: true	Network tab: response has Set-Cookie but browser ignores it? Check console for CORS warning	Add withCredentials: true to axios; ensure CORS credentials: true + explicit origin
2	Cookie set on localhost but not on deployed	Secure: true on HTTP, or domain mismatch	Check cookie Secure flag vs protocol	Set Secure based on NODE_ENV ; ensure HTTPS in prod
3	/auth/refresh returns 401 "No refresh cookie"	Cookie path mismatch between set and refresh endpoint, or SameSite blocking cross-site	Check cookie Path in devtools; confirm request URL matches	Ensure path: "/api/auth" matches the route; switch to SameSite=None if cross-domain

#	Symptom	Likely Cause	Confirm	Fix
4	403 "Forbidden origin" on refresh	allowedOrigins in authController.js doesn't include prod frontend domain	Check req.headers.origin in backend logs	Add prod Vercel URL to allowedOrigins
5	CORS error (preflight fails)	allowedOrigins in server.js doesn't match, or allowedHeaders missing Authorization	Browser console shows CORS error; Network shows failed OPTIONS	Update CORS config in server.js
6	Infinite 401 → refresh loop	isRefreshCall check not matching the URL axios stores	Add console.log(original.url) in interceptor	Ensure original.url?.includes("/auth/refresh") matches
7	authJWT returns 401 on valid token	JWKS fetch fails (Railway can't reach Supabase), or audience mismatch	Backend log: err.code in catch block	Log full error; verify audience : "authenticated" matches token's aud claim; check network egress
8	Login succeeds but /profiles/me returns 401	Token not set in memory before profile call	Add console.log(getAccessToken()) before the profile call	Ensure setAccessToken is called before api.get(...)
9	Refresh works in Postman but not browser	Missing withCredentials ; or SameSite=Lax on cross-site	Check if cookie is sent in browser request (Network → Cookies tab)	Set withCredentials: true ; adjust SameSite
10	clearCookie doesn't clear	path in clearCookie differs from path in res.cookie	Compare both calls	Use identical { path: "/api/auth" } in both

Go / No-Go Criteria

Gate	Pass condition
Cookie present	After login, sb_refresh visible in DevTools as HttpOnly, correct Path, correct SameSite
No localStorage tokens	Zero sb-* keys in Local Storage and Session Storage
Reload recovers	F5 on /dashboard → no login redirect; /auth/refresh returns 200
401 auto-refresh	Expired token → silent refresh → original request succeeds (user sees no error)
Invalid refresh → clean logout	Corrupted cookie → single 401 → redirect to /login , no loop
Logout is terminal	After logout, no cookie, all protected calls fail, no session recovery
CORS clean	No CORS errors in console for any auth flow
Account statuses block	on_hold and inactive users see correct message and are logged out
Prod cookie flags	Secure=true, SameSite appropriate for domain topology
JWKS works from server	authJWT validates tokens without errors on deployed environment

If any gate fails → No-Go. Fix and re-test that gate before proceeding.

Post-Verification: Next Steps (priority order)

1. Lock down route protection

// server.js — protect /api by default, whitelist auth routes

app.use("/api/auth", authRoutes); // public

```

app.use("/api", authJWT, requireActiveProfile); // everything else
app.use("/api/webapp", webAppRoutes); // now protected
app.use("/api/sam", samRoutes); // now protected
// ... etc

```

2. Production cookie config

- If same domain: keep [sameSite: "lax"](#) ✓
- If cross-domain: change to [sameSite: "none"](#) + [secure: true](#)
- Make [allowedOrigins](#) env-driven:

```

const allowedOrigins = process.env.ALLOWED_ORIGINS?.split(",") ||
["http://localhost:5173"];

```

3. Rate limiting

```

npm i express-rate-limit

const rateLimit = require("express-rate-limit");

app.use("/api/auth/refresh", rateLimit({ windowMs: 60000, max: 10 }));

app.use("/api/auth/login", rateLimit({ windowMs: 60000, max: 5 }));

app.use("/api/auth/set-refresh", rateLimit({ windowMs: 60000, max: 5 }));

```

4. Observability — categorized refresh failure logging

```

// In authController.refresh catch block:

const reason = !rt ? "NO_COOKIE" : e.message.includes("revoked") ? "REVOKED" :
"INVALID_REFRESH";

console.error(` [auth] refresh failed: ${reason}`, e.message);

```

5. Environment variable checklist

Variable	Railway	Vercel
NODE_ENV	production	auto
SUPABASE_URL	✓	✓ (VITE_prefix)

Variable	Railway	Vercel
SUPABASE_ANON_KEY	✓	✓ (VITE_ prefix)
SUPABASE_SERVICE_ROLE_KEY	✓	✗ (never expose)
SUPABASE_API_KEY	✓	✗
ALLOWED_ORIGINS	https://<vercel-domain>	n/a
VITE_API_BASE_URL	n/a	https://<railway-domain>/api

-
-
-
-